

C++ FUNDAMENTALS

Dr. S. Vinila Jinny



Arrays within a class

- Arrays can be used as member variables in a class.
- Arrays can be declared as the members of a class.
- The arrays can be declared as private, public or protected members of the class.



```

#include<iostream>
const int size=5;
class student
{
int roll_no;
int marks[size];
public:
void getdata ();
void tot_marks ();
};
void student :: getdata ()
{
cout<<"\nEnter roll no: ";
Cin>>roll_no;
for(int i=0; i<size; i++)
{
cout<<"Enter marks in
subject"<<(i+1)<<": ";
cin>>marks[i] ;
}
}

```

example

```

void student :: tot_marks() //calculating total marks
{
int total=0;
for(int i=0; i<size; i++)
total+ = marks[i];
cout<<"\n\nTotal marks "<<total;
}

void main()
student stu;
stu.getdata() ;
stu.tot_marks() ;
getch();
}

```

Output:

```

Enter roll no: 101
Enter marks in subject 1: 67
Enter marks in subject 2 : 54
Enter marks in subject 3 : 68
Enter marks in subject 4 : 72
Enter marks in subject 5 : 82
Total marks = 343

```



C++ this Pointer

- In C++ programming, **this** is a keyword that refers to the current instance of the class. There can be 3 main usage of this keyword in C++.
- It can be used **to pass current object as a parameter to another method.**
- It can be used **to refer current class instance variable.**
- It can be used **to declare indexers.**

•



C++ this Pointer

```
#include <iostream>
using namespace std;
class Employee {
public:
    int id;
static in x;
    string name;
    float salary;
    Employee(int id, string name, float salary)
    {
        this->id = id;
        this->name = name;
        this->salary = salary;
    }
    void display()
    {
        cout<<id<<" "<<name<<" "<<salary<<endl;
    }
};
```

```
int main(void) {
    Employee e1 =Employee(101, "Sonoo", 890000);
    Employee e2=Employee(102, "Nakul", 59000);
    e1.display();
    e2.display();
    return 0;
}
```

Output:
101 Sonoo 890000
102 Nakul 59000



Static data members in C++

- Static data members are class members that are declared using static keywords.
- A static member has certain special characteristics.
 - Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
 - It is initialized to zero before any object of this class is being created, even before main starts.
 - It is visible only within the class, but its lifetime is the entire program
- Used to maintain values common to the entire class

Syntax

```
static data_type data_member;
```



Static data members (class variables)

```
#include <iostream>

using namespace std;

class Box {
public:
    static int objectCount;

    Box(double l = 2.0, double b = 2.0, double h = 2.0)
    {
        cout << "Constructor called." << endl;
        length = l;
        breadth = b;
        height = h;
        objectCount++;
    }

    double Volume()
    {
        return length * breadth * height;
    }
};
```

```
private:
    double length;
    double breadth;
    double height;
};

int Box::objectCount = 0;

int main(void) {
    Box Box1(3.3, 1.2, 1.5);
    Box Box2(8.5, 6.0, 2.0);
    cout << "Total objects: " << Box::objectCount << endl;
    return 0;
}
```

Output:
Constructor called.
Constructor called.
Total objects: 2



Static Function Members

- By declaring a function member as static, we make it independent of any particular object of the class.
- A static member function can be called even if no objects of the class exist and the **static** functions are accessed using only the class name and the scope resolution operator **::**.
- A static member function can only access static data member, other static member functions and any other functions from outside the class.
- Static member functions have a class scope and they do not have access to the **this** pointer of the class.
- We could use a static member function to determine whether some objects of the class have been created or not.



Static Function Members

```
#include <iostream>
using namespace std;
class Box {
public:
    static int objectCount;
    Box(double l = 2.0, double b = 2.0, double h = 2.0)
    {
        cout << "Constructor called." << endl;
        length = l;
        breadth = b;
        height = h;
        objectCount++;
    }
    double Volume() {
        return length * breadth * height;
    }
    static int getCount()
    {
        return objectCount;
    }
}
```

```
private:
    double length;
    double breadth;
    double height;
};
int Box::objectCount = 0;
int main()
{
    cout << "Initial Stage Count: " << Box::getCount() << endl;
    Box Box1(3.3, 1.2, 1.5);
    Box Box2(8.5, 6.0, 2.0);
    cout << "Final Stage Count: " << Box::getCount() << endl;
    return 0;
}
```

Output:
Initial Stage Count: 0
Constructor called.
Constructor called.
Final Stage Count: 2



Objects as Arguments to Functions

- The objects of a class can be passed as arguments to
 - member functions
 - nonmember functions
 - by value
 - by reference.
- object is passed by value,
 - a copy of the actual object is created inside the function.
 - copy is destroyed when the function terminates.
 - changes made to the copy of the object inside the function are not reflected in the actual object.



Objects as Arguments to Functions

- Objects pass by reference
 - a reference to that object (not the entire object) is passed to the function.
 - the changes made to the object within the function are also reflected in the actual object.
- Whenever an object of a class is passed to a member function of the same class, its data members can be accessed inside the function using the object name and the dot operator.
- Example program:
 - [Object as argument to functions](#)



Friend Functions

- A friend functions in C++ is a function that is declared outside a class but is capable of accessing the private and protected members of the class.
- non-member functions cannot access the private members of a particular class.
- Once declared as a friend function, the function is able to access the private and the protected members of these classes.
- Member functions of one class can be friend function of another class



Declaration of a friend function in C++

```
class class_name  
{  
    friend data_type function_name(arguments/s);  
    //syntax of friend function.  
};
```



Friend function

- `#include <iostream>`
- `using namespace std;`
- `class XYZ`
- `{`
- `private:`
- `int num=100;`
- `char ch='Z';`
- `public:`
- `friend void disp(XYZ obj);`
- `};`
- `void disp(XYZ obj){`
- `cout<<obj.num<<endl;`
- `cout<<obj.ch<<endl;`
- `}`

```
int main()
{
    XYZ obj;
    disp(obj);
    return 0;
}
```



Characteristics of Friend Function in C++

- The function is not in the 'scope' of the class to which it has been declared a friend.
- Friend functionality is not restricted to only one class
- It cannot be invoked using the object as it is not in the scope of that class.
- Friend functions have objects as arguments.
- It cannot access the member names directly and has to use dot membership operator and use an object name with the member name.
- We can declare it either in the 'public' or the 'private' part.



Member function as friend

```
class X
{
....int fun1();
....
};
class y
{
....
friend int X::func1();
....
};
```



Friend Class

- A friend class can access both private and protected members of the class in which it has been declared as friend.



Example

```
#include <iostream>
using namespace std;
class A
{
    int x=4;
    friend class B; //friend class
};
class B
{
    public:
    void display (A &a)
    {
        cout<<"value of x is:" <<a.x;
    }
};
```

```
int main ()
{
    A a;
    B b;
    b. display (a);
    return 0;
}
```

Output:
value of x is:4



About Friend Class

- If class A is a friend of class B, then class B is not a friend of class A.
- Also, if class A is a friend of class B, and then class B is a friend of class C, class A is not a friend of class C.
- If Base class is a friend of class X, subclass Derived is not a friend of class X; and if class X is a friend of class Base, class X is not a friend of subclass Derived.

